

Name: Sakshi Padole Class: Soc-08 Batch: B
Subject: Python (P.I) Roll No.: 37 Exp. No.: 01
Name of the Experiment: Introduction to python programming
Performed on: 10/01/2026
Submitted on: 30/01/2026

Marks
10

Teacher's Signature
with date

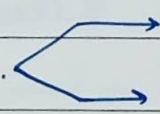
Q.1) list the key features and advantages of python.
Also mention any two python IDEs or editors used for program development.

→ key features of python -

1. it is simple and easy-to-learn programming language.
2. it is independent language, so code is executed line by line.
3. it is open source language and easy to use.
4. Python has a large standard library.

• Advantages of python -

1. Require less code compare to other language.
2. Easy debugging and maintenance.
3. Use in web development and data science.
4. Suitable for beginners.

Python IDEs/Editors.  Pycharm
visual studio code (vs)

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐

Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐

Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐

Teacher's Signature

Q.2) Explain the history and evolution of python describe way python has become popular for beginners and industry applications:

-
- The history and evolution of python -
 - python was created by Guido Van Rossum in 1991
 - Named after Monty python's Flying circus.
 - Developed to emphasize code readability.
 - Python 2 released in 2000
 - Python 3 released in 2008 (more powerful & secure)
- Why python is popular -
 - it is easy syntax for beginners.
 - Used in industry.
 - Strong community support.
 - Libraries for AI, ML, Web, Automation.

Q.3) Write a python program to accept two numerical value from the user perform the following operations:

→ program :-

```
a = float(input("Enter First number"))  
b = float(input("Enter second number"))  
print("Addition =", a+b)  
print("Subtraction =", a-b)  
print("Multiplication =", a*b)  
print("Division =", a/b)
```

Explanation - The program accepts two no. values from the user and performs addition, subtraction, multiplication, division.

Q.4) Analyze the python code and identify data types.
Explain the output.

→
a=10 a → Integer
b=5.5 b → Float
c="python" c → String
d=True d → Boolean

Data types.

Explanation - python automatically assigns data types based on the value stored in the variables.

Q.5) Evaluate relational, logical, bitwise operators with examples where each type of operator is most appropriate.

→

a) Relational operators -

Used to compare values

code:- a=10

b=5

print(a > b)

Used in decision-making statements.

b) Logical operators -

Used to combine multiple conditions.

code:- x=True

y=False

print(x and y)

Used in conditional logic.

c) Bitwise operators -
used in bit-level operations.

```
code:- a=5  
       b=3
```

```
print(a & b)
```

Used in low level programming.

Q.6) Design and implement a menu-driven python program that....

- Accepts user input.
- performs user arithmetic, relational, logic, and assignment operation.
- Displays results should handle int, float, boolean data types.

→ code:-

```
a = int(input("enter first number:"))  
b = int(input("enter second number:"))
```

```
print("1. Arithmetic operation")
```

```
print("2. Relational operation")
```

```
print("3. Logical operation")
```

```
print("4. Assignment operation")
```

```
choice = int(input("Enter your choice:"))
```

```
if choice == 1:
```

```
    print("Addition =", a+b)
```

```
elif choice == 2:
```

```
    print("a > b =", a > b)
```

```
elif choice == 3:
```



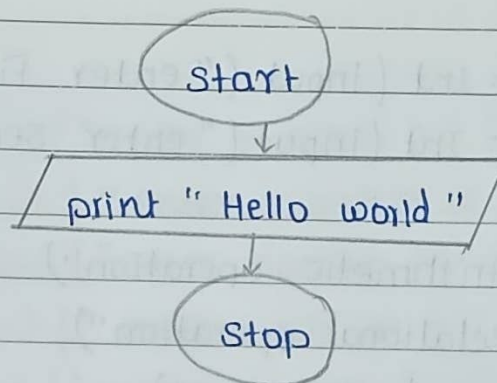
```
print ((a>0) and (b>0))  
elif choice == 4:  
    at = 6  
print ("result after assignment :", a)  
else:  
    print ("Invalid choice").
```

Q.7) Write a program in python to display "Hello world".

→ Algorithm :

1. start
2. print "Hello world"
3. Stop.

Flow chart :



python code : `print("Hello world")`

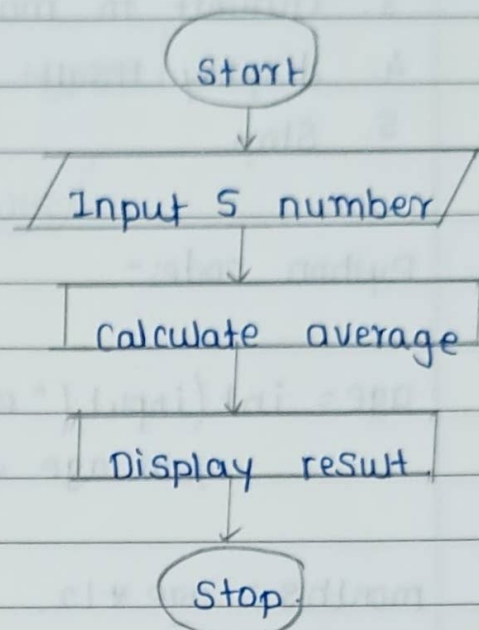
Output: Hello world.

Q.8) Write a program to perform average of five numbers using input() function and type conversion concept:

→ Algorithm:-

Flow chart:-

1. Start.
2. Input Five numbers
3. Convert to Float
4. Calculate average
5. Display result.
6. Stop.



Code:-

```
a = float (Input ("enter number 1:"))
b = float (Input ("enter number 2:"))
c = float (Input ("enter number 3:"))
d = float (Input ("enter number 4:"))
e = float (Input ("enter number 5:"))
```

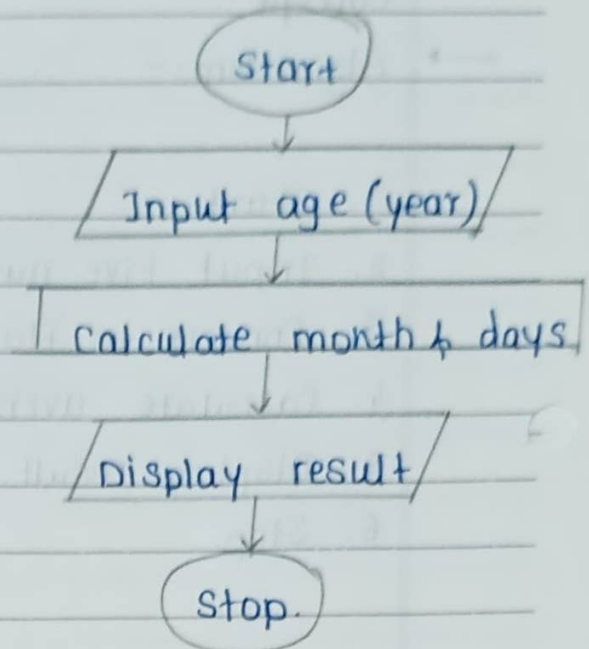
```
avg = (a+b+c+d+e)/5
print ("Average =", avg)
```

Q.9) Write a program that prompts the user to enter their age and prints out their age in years, month, days

if choice == ti

- Algorithm -
- Flow chart :-

1. Start
2. Input age in year
3. Convert to months & day
4. display result
5. Stop.



- Python code:-

```
age = int(input("enter  
your age years:"))
```

```
months = age * 12
```

```
days = age * 365
```

```
print("Age in years:", age)
```

```
print("Age in month:", month)
```

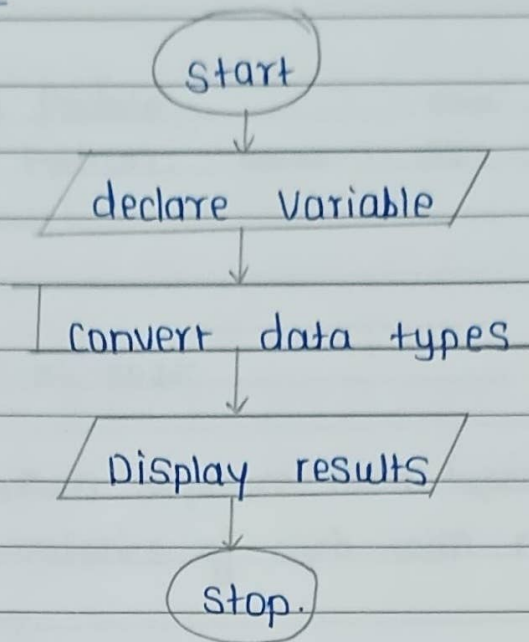
```
print("Age in day:", days)
```

Q.10) Write a program to demonstrate the concept of data types & conversion.

→ Algorithm:-

1. Start.
2. Declare variables of different data types.
3. Convert data types.
4. Display values.
5. Stop.

Flow chart:-



python code:-

```
a=10
b=5.5
c="20"
print (type (a))
print (type (b))
print (type (c))

c= int (c)
print ("After conversion:", c)
print (type (c))
```




Name: Sakshi Padole Class: SOC-08 Batch: B

Subject: PL- Python Roll No.: 37 Exp. No.: 02

Name of the Experiment: _____

Performed on: _____

Submitted on: 17-02-2026

Marks	Teacher's Signature with date
10	

Q.1] List the python sequence data types and state one key characteristics of each with respect to mutability or ordering.

→ In python, sequence data types store collections of items in a specific order.

1. List - ordered sequence
Mutable (elements can be changed after creation)

2. Tuple:- ordered sequence
Immutable (cannot be changed once created)

3. String:- ordered sequence of characters.
Immutable (characters cannot be modified)

4. Range:- ordered sequence of numbers.
Immutable (cannot modify values, only generate them.)

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐

Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐

Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐

Teacher's Signature

Q.2 Explain how indexing and slicing work in python sequences illustrate how they differ when applied to strings and lists.

→ Indexing in python sequences.

indexing is used to access a single element from a sequence.

- Indexing starts from 0 (left to right).
- Negative indexing starts from -1 (right to left)

Ex:- code:

```
S = "python"
```

```
l = [10, 20, 30, 40]
```

```
print(S[0])
```

```
print(S[-1])
```

```
print(l[1])
```

```
print(l[-2])
```

- Slicing in python sequences.

slicing is used to extract a part (sub sequences) of a sequence.

- Syntax - sequence (start - stop - step)

Start → Starting index (included)

Stop → ending index (excluded)

step → gap between elements (optional)


```
Ex - print (s[1:9])  
      print (l[0:3])  
      print (s[::2])  
      print (l[::-1])
```

Q3] Write a python program that:

- Uses indexing and slicing on a string, list and applies at least two built in function to display meaningful result.

→ python code -

```
Text = "python programming"
```

Indexing.

```
first - char = text[0]
```

```
last - char = text[-1]
```

Slicing

```
sub - text = text[0:6]
```

```
every - second = text[::2]
```

```
length = len(text)
```

```
max - char = max(text)
```

```
print("original string:", text)
```

```
print("First character:", first - char)
```

```
print("Last character:", last - char)
```

```
print("Sliced string (0:6):", sub - text)
```

```
print("Every second character:", every - second)
```

```
print("length of string:", length)
```

Q.4) Analyze the difference between equality (`==`) and identity (`is`) when used with sequence types discuss how object mutability and memory reference affect program behaviour.

→ Difference between `==` and `is` in python.

- The equality operator (`==`) compares the values or contents of two sequence objects.
- The identity operator (`is`) compares the memory location of two objects.
- Two sequences may have the same values but be stored in different memory locations, so `==` returns true while `is` returns false.
- For mutable sequence like lists if two variables refer to the same object, changes made through one variable affect the other.
- For immutable sequence like string and tuples, values cannot be modified, any change creates a new object, reducing side effects.

Q.5) Evaluate the suitability of lists, sets and dictionaries for storing and managing unique student records. Justify which data structure is most efficient and why.

→ Suitability of lists, sets and dictionaries for unique student records.

- When storing and managing unique student records the choice of data structure affects efficiency, data integrity and ease of access.

1. Lists -

- Lists store elements in an ordered sequence.
- They allow duplicate entries, so extra logic is required to ensure uniqueness.
- Searching for a student record is slow ($O(n)$) especially for large datasets.

2. Sets -

- Sets store only unique elements automatically.
- They are unordered and do not support indexing.
- Fast membership checking ($O(1)$ average time)

3. Dictionaries.

- Dictionaries store data as key-value pairs.
- Keys are unique ensuring no duplicate student IDs.
- Very fast across insertion and deletion ($O(1)$ average time).

Q.6) Design and implement a python program that is.

→ Appropriate Sequence types

→ Indexing or Slicing is built in functions to manage and process student data efficiently.

→ students = {

1: {"name": "Amit", "marks": [78, 85, 90]}

2: {"name": "Sneha", "marks": [88, 91, 81]}

}

for roll, data in students.items():

marks = data["marks"]

```
print(F"\n Roll NO: {roll}")  
print(F"name: {data('name')}")  
print(F"marks {marks}")  
print(F" First marks: {first - mark}")  
print(F" Top two marks: {top - two}")
```

Indexing (marks [0] and slicing [:2]) are used
Built in functions: sum(), len(), sorted().

Q.7] Write a program to print the first and last character of a string.

→

```
text = input("enter a string:")
```

```
print("First character:", text[0])
```

```
print("last character:", text[-1])
```

- text[0] → accesses the first character.
text[-1] → accesses the last character using negative indexing.

Q.8] Write a program to create a list, tuple, set, and dictionary.

→

```
my_list = [10, 20, 30, 40]
```

```
my_tuple = (1, 2, 3, 4)
```

```
my_set = {5, 10, 15}
```



```
my — dict = { 5, 10, 15 }
```

```
my — dict = {  
    "name": "Amp",  
    "age": 20,  
    "branch": "CSF"  
}
```

```
print ("list:" my — list)  
print (" Tuple:", my — tuple)  
print ("set:", my — set)  
print (" Dictionary", my — dict),
```

Q.9] Write a program to understand the concept of indexing.

→ text = "python"

```
print (" string:", text)  
print (" First character:", text[0])  
print (" second character:", text[1])  
print (" last character", text[-1])
```

```
numbers = [10, 20, 30, 40, 50]
```

```
print (" In first list:" numbers)
```

Q.10] Write a program to understand the concept of built in function.

→ numbers = [10, 20, 80, 40, 50]

```
print("list:", numbers)
```

```
print("length of list:", len(numbers))
```

```
print("maximum value:", max(numbers))
```

```
print("minimum value:", min(numbers))
```

```
print("sum of elements:", sum(numbers))
```

```
print("sorted list:", sorted(numbers)).
```

